

Лабораторная работа №2

Исследование протокола TCP и алгоритма управления очередью RED

Шубнякова Дарья НКНбд-01-22

1. Вводная часть

2. Основная часть

3. Результаты

4. Результаты

1. Вводная часть

1.1 Цели и задачи

- Исследовать протокол TCP и алгоритм управления очередью RED. Требуется разработать сценарий, реализующий модель, построить в Xgraph график изменения TCP-окна, график изменения длины очереди и средней длины очереди. – Изменить в модели на узле s1 тип протокола TCP с Reno на NewReno, затем на Vegas. Сравните и поясните результаты. – Внести изменения при отображении окон с графиками (изменить цвет фона, цвет траекторий, подписи к осям, подпись траектории в легенде).

2. Основная часть

2.1 Выполнение лабораторной работы

Создаем нужную нам модель с дуплексными узлами. Берем алгоритм из лабораторной работы.

```
Терминал - openmodelica@openmodelica-VirtualBox: ~/mip/lab-ns
Файл  Правка  Вид  Терминал  Вкладки  Справка
GNU nano 2.9.3 example5.tcl

set ns [new Simulator]
set nf [open out.nam w]
$ns namtrace-all $nf
set f [open out.tr w]
$ns trace-all $f

# Узлы сети:
set N 5
for {set i 1} {$i < $N} {incr i} {
    set node_($i) [$ns node]
}
set node_r1 [$ns node]
set node_r2 [$ns node]

# Соединения:
$ns duplex-link $node_(s1) $node_r1 10Mb 2ms DropTail
$ns duplex-link $node_(s2) $node_r1 10Mb 3ms DropTail
$ns duplex-link $node_r1 $node_r2 1.5Mb 20ms RED
$ns queue-limit $node_r1 $node_r2 25
$ns queue-limit $node_r2 $node_r1 25
$ns duplex-link $node_(s3) $node_r2 10Mb 4ms DropTail
$ns duplex-link $node_(s4) $node_r2 10Mb 5ms DropTail

# Агенты и приложения:
set tcp1 [$ns create-connection TCP/Reno $node_(s1) TCPSink $node_(s3) 0]
$tcp1 set window_ 15
set tcp2 [$ns create-connection TCP/Reno $node_(s2) TCPSink $node_(s3) 1]
$tcp2 set window_ 15
set ftp1 [$tcp1 attach-app FTP]
set ftp2 [$tcp2 attach-app FTP]
```

2.2 Вот так будет выглядеть процедура финиш.

```
# Процедура finish:
proc finish {} {
    global ns nf f tchan_ windowVsTime

    $ns flush-trace
    close $f
    close $nf
    close $windowVsTime

    if { [info exists tchan_] } {
        close $tchan_
    }
}
```

Рисунок 2: Процедура

```

# подключение кода Awk:
set awkCode {
    {
        if ($1 == "Q" && NF>2) {
            print $2, $3 >> "temp.q";
        }
        else if ($1 == "a" && NF>2) {
            print $2, $3 >> "temp.a";
        }
    }
}

set f [open temp.queue w]
puts $f "TitleText: red"
puts $f "Device: Postscript"

exec rm -f temp.q temp.a
exec touch temp.a temp.q

# выполнение кода Awk
catch {exec awk $awkCode all.q}

puts $f "\"queue"
catch {exec cat temp.q >@ $f}
puts $f \n
puts $f "\"ave_queue"
catch {exec cat temp.a >@ $f}
close $f

```


2.4 И пишем часть для отображения графиков в Xgraph.

```
# Запуск xgraph с графиками окна TCP и очереди:  
exec xgraph -bb -tk -x time -t "TCP Reno Cwnd" WindowVsTimeReno &  
exec xgraph -bb -tk -x time -y queue temp.queue &  
  
# Запуск nam  
exec nam out.nam &  
  
exit 0  
}  
  
$ns run
```

Рисунок 3: Отображение графиков

2.5 Так будет выглядеть график динамики размера окна TCP Reno.

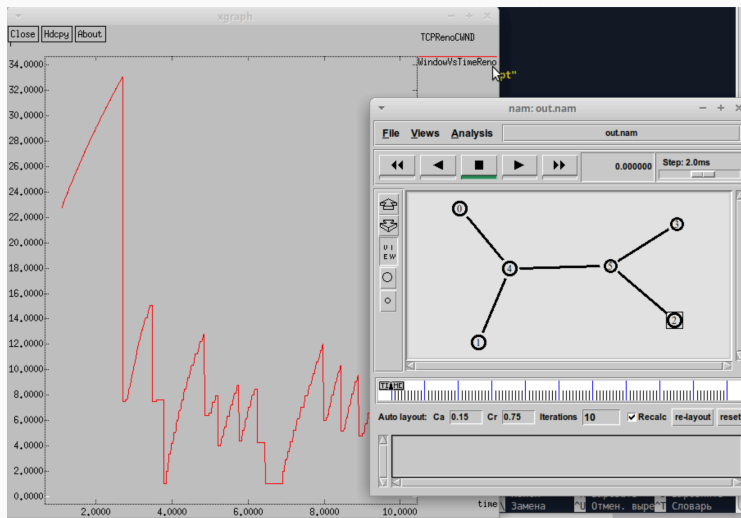


Рисунок 4: График динамики размера окна TCP Reno

2.6 А здесь мы видим график динамики длины очереди и средней длины очереди.

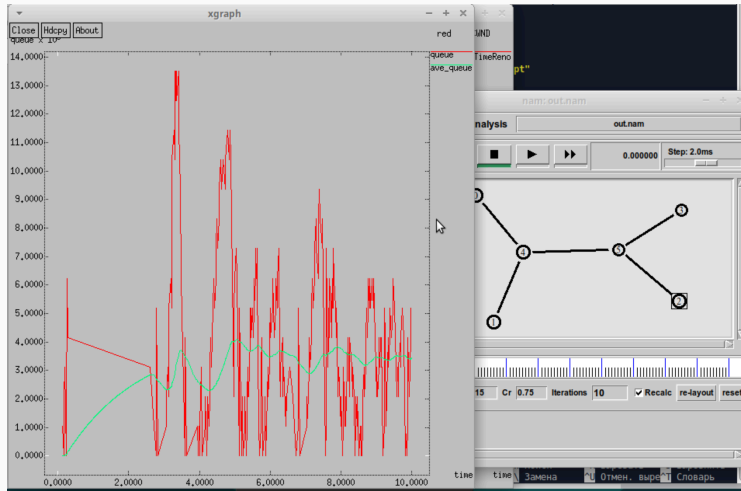


Рисунок 5: График очередей

2.7 Меняем на алгоритм NewReno, важно соблюдать именно такое написание, как на скриншоте.

```
# Агенты и приложения:  
set tcp1 [$ns create-connection TCP/Newreno $node_(s1) TCPSink $node_$  
$tcp1 set window_ 15  
set tcp2 [$ns create-connection TCP/Reno $node_(s2) TCPSink $node_(s3$  
$tcp2 set window_ 15  
set ftp1 [$tcp1 attach-app FTP]  
set ftp2 [$tcp2 attach-app FTP]
```

Рисунок 6: TCP NewReno

2.8 Меняем цвет линий графика, фон графика, слегка меняем подписи осей, легенду и названия окон, чтобы показать освоение навыков в Xgraph.

```
set f [open temp.queue w]
puts $f "TitleText: RED"
puts $f "Device: Postscript"
    puts $f "0.Color: Blue"
    puts $f "1.Color: Orange"

exec rm -f temp.q temp.a
exec touch temp.a temp.q

# выполнение кода AWK
catch {exec awk $awkCode all.q}

puts $f "\"current_queue"
catch {exec cat temp.q >@ $f}
puts $f \n
puts $f "\"average_queue"
catch {exec cat temp.a >@ $f}
close $f

# Запуск xgraph с графиками окна TCP и очереди:
exec xgraph -bb -tk -x TIME -t "TCPNewRenoCWND" WindowVsTimeNewReno -bg$
exec xgraph -bb -tk -x TIME -y QUEUE temp.queue -bg white &
```

Рисунок 7: Изменения

2.9 Меняем так же цвет линии в графике динамики размера окна TCP.

```
# Мониторинг размера окна TCP:  
set windowVsTime [open WindowVsTimeNewReno w]  
puts $windowVsTime "0.Color: Blue"  
set qmon [$ns monitor-queue $node_(r1) $node_(r2) [open qm.out w] 0.1]  
[$ns link $node_(r1) $node_(r2)] queue-sample-timeout
```

Рисунок 8: Изменения

2.10 Видим, что все изменения были применены, и оцениваем график TCP NewReno.

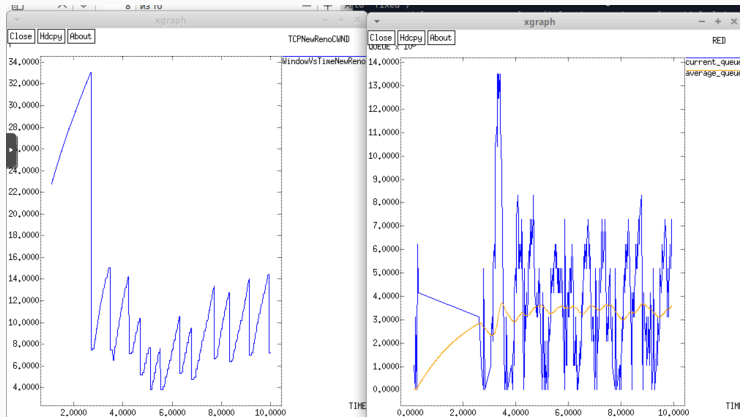


Рисунок 9: Графики для TCP NewReno

2.11 Меняем цвет линии на графике динамики размера окна TCP и на первом узле ставим TCP Vegas.

```
# Агенты и приложения:
set tcp1 [$ns create-connection TCP/Vegas $node_(s1) TCPSink $node_(s$
$tcp1 set window_ 15
set tcp2 [$ns create-connection TCP/Reno $node_(s2) TCPSink $node_(s3$
$tcp2 set window_ 15
set ftp1 [$tcp1 attach-app FTP]
set ftp2 [$tcp2 attach-app FTP]

# Мониторинг размера окна TCP:
set windowVsTime [open WindowVsTimeVegas w]
puts $windowVsTime "0.Color: Purple"
set qmon [$ns monitor-queue $node_(r1) $node_(r2) [open qm.out w] 0.1]
[$ns link $node_(r1) $node_(r2)] queue-sample-timeout
```

Рисунок 10: Изменения

2.12 И меняем тайтлы и цвета для графика средней и текущей очередей.

```
set f [open temp.queue w]
puts $f "TitleText: RED"
puts $f "Device: Postscript"
    puts $f "0.Color: Pink"
    puts $f "1.Color: Grey"

exec rm -f temp.q temp.a
exec touch temp.a temp.q

# выполнение кода AWK
catch {exec awk $awkCode all.q}

puts $f "\"current_queue"
catch {exec cat temp.q >@ $f}
puts $f \n
puts $f "\"average_queue"
catch {exec cat temp.a >@ $f}
close $f

# Запуск xgraph с графиками окна TCP и очереди:
exec xgraph -bb -tk -x TIME -t "TCPVegasCWND" WindowVsTimeVegas -> $
exec xgraph -bb -tk -x TIME -y QUEUE temp.queue -bg white &

# Запуск nam
exec nam out.nam &
```

2.13 Оцениваем работу при TCP Vegas.

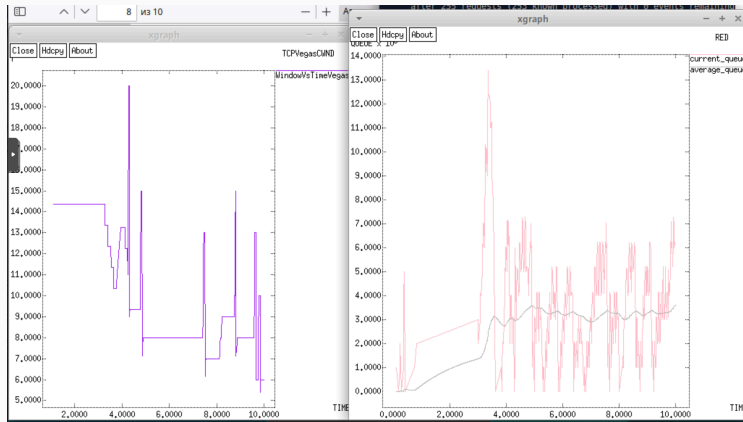


Рисунок 12: Графики TCP Vegas

3. Результаты

3. Результаты

Графики динамики размера окна у Reno и NewReno довольно похожи, второй действует чуть более плавно и нет таких сильных разбросов на протяжении работы. График динамики размера окна TCP Vegas очень ступенчатый, перепады резкие, нет плавных линий. В сети с RED очередью: - Reno: постоянно “прощупывает” пределы пропускной способности - Vegas: находит оптимальный уровень и держится near него - При изменении условий Vegas делает шаг к новому оптимальному уровню

4. Результаты

4. Результаты

График средней и текущей очередей у NewReno и Reno опять же похожи, но текущая очередь у NewReno сразу начинает выравниваться после пика, а у Reno это происходит постепенно. График Vegas опять же в отличие от первых двух начинается с падения. TCP Reno/NewReno: - Текущая очередь: резкие пики и провалы, хаотичные колебания - Средняя очередь: менее изменчивая, но все равно волнистая - Причина: реагируют только на потери, постоянно “перескакивают” оптимальный уровень TCP Vegas: - Текущая очередь: более стабильная, плавные изменения - Средняя очередь: почти прямая линия near оптимального уровня - Причина: proactively регулирует окно на основе задержки, предотвращает переполнение очереди